Research Note

The Learner Problem

Low-Level Design (LLD) practice is a critical skill for software engineering interviews and real-world system building. However, it suffers from a fundamental evaluation gap: **it is easy to start but hard to evaluate**.

A learner can sketch a Parking Lot, Elevator, or Vending Machine design on a whiteboard or in a text file and feel confident about their solution. But they have no way to know:

- Whether their class responsibilities are well-separated (SRP violations? Too many responsibilities in one class?)
- Whether they are using appropriate design patterns or over-engineering
- Whether their relationships between classes (inheritance vs. composition) are sound
- What trade-offs they have — or haven't — considered

This leads to repeated practice without meaningful feedback, which is one of the most effective ways to improve at LLD.

Existing Approaches & Tools Researched

1. InterviewBit / LeetCode Premium LLD Sections

- **What**: Provide LLD problem statements and sample solutions.
- **Gap**: No feedback mechanism. Learners submit their own solutions to mentors or discuss in forums. Qualitative and asynchronous.

2. Grokking the Low-Level Design Interview (Educative)

- **What**: Step-by-step walkthrough of LLD problems with model solutions.
- **Gap**: Passive consumption. No interactive practice loop. No personalization of feedback.

3. Pramp / InterviewBuddy

- **What**: Mock interviews with peers or hired interviewers who give verbal feedback.
- **Gap**: Human feedback is expensive, inconsistent, and requires scheduling. Not scalable for repeated practice.

4. Excalidraw + Mermaid /draw.io

- **What**: Diagramming tools for designing class diagrams.
- **Gap**: Good for drawing but provide zero design feedback. No integration with evaluation.

5. CodeSignal / HackerRank LLD-like Problems

- **What**: Primarily algorithmic; some system design questions.
- **Gap**: No true LLD design-and-evaluate cycle. No class-level feedback.

Key Gaps Identified

Gap	Description
No Automated Feedback	No tool provides immediate, structured feedback on LLD solutions. Learners rely on human mentors.
No Practice History	Most tools don't track repeated attempts, missing the improvement-over-time dimension.
No Design Trade-off Analysis	Tools don't prompt learners to consider or evaluate design alternatives.
No Multi-modal Submission	Learners want to submit text, code, and diagrams, but tools typically support only one format.
No Deterministic Baseline	Feedback (when available) is entirely subjective, with no reproducible baseline for comparison.

Product Direction

Build a **focused LLD practice platform** that implements a tight feedback loop:

```
Choose Problem → Think/Design → Submit → Get Feedback → Review → Try Again
```

MVP Scope

- A curated catalog of 6 LLD problems (Parking Lot, Elevator, Vending Machine, Library, Tic-Tac-Toe, LRU Cache)
- Submission in multiple formats (text, code, diagram, mixed)
- **Hybrid feedback engine**: deterministic rule-based scoring (structural + keyword analysis) combined with simulated LLM-based qualitative feedback
- Attempt history showing all past attempts per problem
- Extensibility via the Strategy pattern for adding new feedback providers and submission formats

Why This Approach

1. **Deterministic feedback** provides a reproducible baseline (structural checks like class count, pattern usage)
2. **LLM feedback** adds qualitative depth (trade-off analysis, pattern appropriateness) that pure rules can't
3. **History tracking** turns one-time problem solving into iterated improvement
4. **Clean architecture** makes the platform's own design an example of good LLD, reinforcing the learning objective